

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3-4.

Вариант 3312423344

Выполнила

Розова Елена Алексеевна

Р3110

Проверил

Чупанов Аликылыч Алибекович

Преподаватель факультета ПИиКТ

г. Санкт-Петербург 2025г

Содержание

UML - таблица	4
Мой код	5
Метод от ИИ:	19
Заключение	24

Задание

В соответствии с выданным вариантом на основе предложенного текстового отрывка из литературного произведения создать объектную модель реального или воображаемого мира, описываемого данным текстом. Должны быть выделены основные персонажи и предметы со свойственным им состоянием и поведением. На основе модели написать программу на языке Java.

Текст:

И вот на площадке появились журавли - большие серые птицы на длинных стройных ногах, с гибкой шеей, с красным хохолком на маленькой точеной головке. Широко раскрыв крылья, журавли то взлетали, то, едва коснувшись земли, быстро кружились на одной ноге. Казалось, на площадке мелькают не птицы, а серые тени. Кто научил журавлей скользить так легко и бесшумно? Может быть, туман, стелющийся над болотами? Может быть, вольный ветер, проносящийся над землей? Или облака, проплывающие в небе? Все на Кулаберге, словно замороженные, следили за журавлями. Птицы тихонько поднимали и опускали крылья, звери покачивались из стороны в сторону: одни - похлопывали хвостами в лад журавлиному танцу, другие - наклоняли рога.

Требования к объектной модели, сценарию и программе:

В модели должны быть представлены основные персонажи и предметы, описанные в исходном тексте. Они должны иметь необходимые атрибуты и характеристики (состояние) и уметь выполнять свойственные им действия (поведение), а также должны образовывать корректную иерархию наследования классов.

Объектная модель должна реализовывать основные принципы ООП - инкапсуляцию, наследование и полиморфизм. Модель должна соответствовать принципам SOLID, быть расширяемой без глобального изменения структуры модели.

Сценарий должен быть вариативным, то есть при изменении начальных характеристик персонажей, предметов или окружающей среды, их действия могут изменяться и отклоняться от базового сценария, приведенного в исходном тексте. Кроме того, сценарий должен поддерживать элементы случайности (при генерации персонажей, при задании исходного состояния, при выполнении методов).

Объектная модель должна содержать как минимум один корректно использованный элемент каждого типа из списка:

абстрактный класс как минимум с одним абстрактным методом;

интерфейс;

перечисление (enum);

запись (record);

массив или ArrayList для хранения однотипных объектов;

проверяемое исключение.

В созданных классах основных персонажей и предметов должны быть корректно переопределены методы equals(), hashCode() и toString(). Для классов-исключений необходимо переопределить метод getMessage().

Созданные в программе классы-исключения должны быть использованы и обработаны. Кроме того, должно быть использовано и обработано хотя бы одно unchecked исключение (можно свое, можно из стандартной библиотеки).

При необходимости можно добавить внутренние, локальные и анонимные классы.

UML - таблица

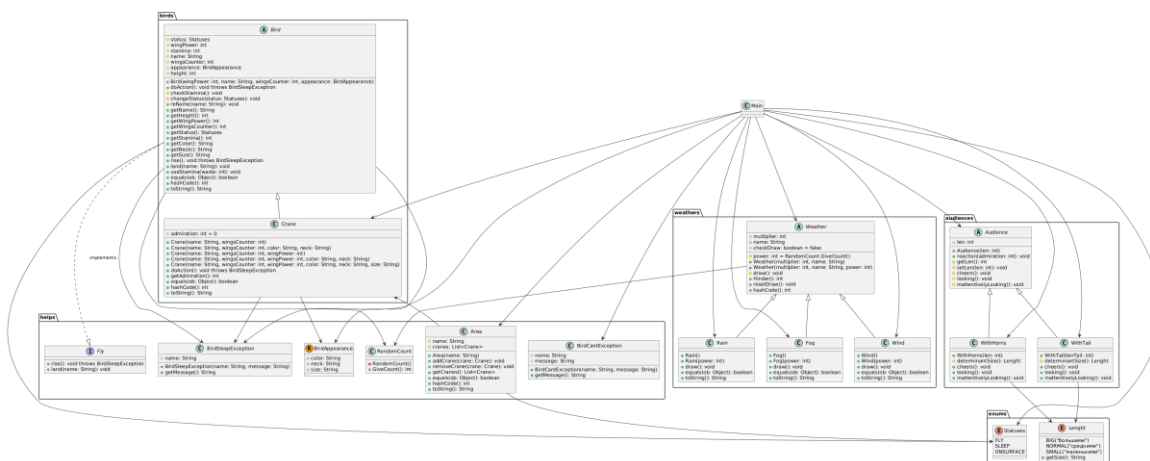


Рисунок 1 – uml-таблица

Мой код

```
public abstract class Bird implements Fly { 2 usages 1 inheritor 1 alien *
    private Statuses status = Statuses.ONSURFACE; 11 usages
    private final int wingPower; 6 usages
    private int stamina; 10 usages
    private String name = "Птица"; 10 usages
    private int wingsCounter = 2; 5 usages
    private final BirdAppearance appearance; 5 usages
    private int height = 0; 4 usages

    Bird(int wingPower, String name, int wingsCounter, BirdAppearance appearance) {
        this.wingPower = wingPower;
        this.stamina = wingPower * RandomCount.GiveCount();
        this.name = name;
        this.wingsCounter = wingsCounter;
        this.appearance = appearance;
    }

    public abstract void doAction() throws BirdSleepException; 1 usage 1 implementation

    protected void checkStamina() { 3 usages 1 alien
        if (stamina <= 0 && status != Statuses.SLEEP) {
            this.status = Statuses.SLEEP;
            System.out.println(name + " устал и больше не может продолжать");
        }
    }

    protected void changeStatus(Statuses status) { this.status = status; }

    public void reName(String name) { this.name = name; }

    public String getName() { return name; }

    public int getHeight() { 2 usages new *
        return height;
    }

    public int getWingPower() { no usages new *
        return wingPower;
    }

    public int getWingsCounter() { no usages new *
        return wingsCounter;
    }

    public Statuses getStatus() { 2 usages new *
        return status;
    }

    public int getStamina() { return stamina; }

    public String getColor() { return appearance.color(); }

    public String getNeck() { return appearance.neck(); }

    public String getSize() { return appearance.size(); }

    @Override 2 usages 1 alien
    public void rise() throws BirdSleepException {
        if (wingsCounter < 2) {
            changeStatus(Statuses.SLEEP);
        }
    }
}
```

Рисунок 2 - абстрактный класс Bird

```

@Override 2 usages  ⚡ elen
public void rise() throws BirdSleepException {
    if (wingsCounter < 2) {
        changeStatus(States.SLEEP);
        throw new BirdSleepException(name, "не может взлететь - недостаточно крыльев");
    }

    int waste = stamina / 10 + 1;
    useStamina(-(waste));
    checkStamina();
    if (!status.equals(States.SLEEP)) {
        changeStatus(States.FLY);
        height += 50;
        System.out.println("Ироко раскрыв крылья " + name + " взлетел");
    }
}

@Override 1 usage  ⚡ elen
public void land(String name) {
    height = 0;
    int waste = stamina / 10 + 1;
    useStamina(-(waste));
    checkStamina();
    if (!status.equals(States.SLEEP)) {
        status = States.ONSURFACE;
        System.out.println(name + " коснулся земли и крутится на одной ноге");
    }
}

public void useStamina(int waste) { stamina -= waste; }

@Override 4 usages 1 override  ⚡ elen
public boolean equals(Object ob) {
    if (this == ob) {
        return true;
    }
    if (ob == null || getClass() != ob.getClass()) {
        return false;
    }
    Bird crane = (Crane) ob;
    return Objects.equals(status, crane.status) && Objects.equals(wingPower, crane.wingPower) &&
}

@Override no usages 1 override  ⚡ elen
public int hashCode() { return Objects.hash(stamina, status, name, wingPower, wingsCounter); }

@Override 1 override  ⚡ elen
public String toString() {
    return "Crane{"
        + "status=" + status
        + ", wingPower=" + wingPower
        + ", stamina=" + stamina
        + ", name=" + name
        + ", appearance=" + appearance
        + ", height=" + height
        + ", wingCountent=" + wingsCounter + "}";
}

```

Рисунок 3 - Bird

```

public class Crane extends Bird { 20 usages & elen
    protected int admiration = 0; 7 usages

    public Crane(String name, int wingsCounter) { 1 usage & elen
        super(wingPower: 2, name, wingsCounter, new BirdAppearance(color: "серая", neck: "длинная", size: "большая"));
    }

    public Crane(String name, int wingsCounter, String color, String neck) { no usages & elen
        super(wingPower: 2, name, wingsCounter, new BirdAppearance(color, neck, size: "большая"));
    }

    public Crane(String name, int wingsCounter, int wingPower) { 1 usage & elen
        super(wingPower, name, wingsCounter, new BirdAppearance(color: "серая", neck: "длинная", size: "большая"));
    }

    public Crane(String name, int wingsCounter, int wingPower, String color, String neck) { 1 usage & elen
        super(wingPower, name, wingsCounter, new BirdAppearance(color, neck, size: "большая"));
    }

    public Crane(String name, int wingsCounter, int wingPower, String color, String neck, String size) { no usages & elen
        super(wingPower, name, wingsCounter, new BirdAppearance(color, neck, size));
    }

    @Override 1 usage & elen
    public void doAction() throws BirdSleepException {
        if (getHeight() == 0) {
            rise();
            admiration += 1;
        } else if (getHeight() > 0) {
            land(getName());
            admiration += 1;
        }
        checkStamina();
    }

    public int getAdmiration() { return admiration; }

    @Override 4 usages & elen
    public boolean equals(Object ob) {
        if (this == ob) {
            return true;
        }
        if (ob == null || getClass() != ob.getClass()) {
            return false;
        }
        Crane crane = (Crane) ob;
        return Objects.equals(admiration, crane.admiration);
    }

    @Override no usages & elen
    public int hashCode() {
        return Objects.hash(admiration);
    }

    @Override & elen
    public String toString() { return "Crane{, admirations=" + admiration + "}"; }
}

```

Рисунок 4 - дочерний класс Crane

```

package weathers;

import helps.RandomCount;

import java.util.Objects;

public abstract class Weather { 6 usages 3 inheritors 2 elen *
    protected int multiplier; 13 usages
    public String name;
    protected boolean checkDraw = false; 3 usages
    protected int power = RandomCount.GiveCount(); 2 usages

    Weather(int multiplier, String name) { 3 usages 2 elen *
        this.multiplier = multiplier;
        this.name = name;
    }

    Weather(int multiplier, String name, int power) { 3 usages 2 elen *
        this.multiplier = multiplier;
        this.name = name;
        this.power = power;
    }

    abstract void draw(); 1 usage 3 implementations 2 elen

    public int Hinder() { 2 usages 2 elen *
        if (!checkDraw) {
            draw();
            checkDraw = true;
        }
        return multiplier * power / 10;
    }

    public void resetDraw() { checkDraw = false; }

    @Override no usages 2 elen *
    public int hashCode() { return Objects.hash(name, multiplier); }
}

```

Рисунок 5 - абстрактный класс погоды Weather

```

package weathers;

import java.util.Objects;

public class Wind extends Weather { 5 usages 2 elen *
    public Wind() { super( multiplier: 1, name: "Ветер"); }

    public Wind(int power) { super( multiplier: 1, name: "Ветер", power); }

    @Override 1 usage 2 elen
    void draw() { System.out.println("Вдуг подул ветер."); }

    @Override 4 usages 2 elen *
    public boolean equals(Object ob) {
        if (this == ob) {
            return true;
        }
        if (ob == null || getClass() != ob.getClass()) {
            return false;
        }
        Wind wind = (Wind) ob;
        return Objects.equals(multiplier, wind.multiplier) && Objects.equals(name, wind.name);
    }

    @Override 2 elen *
    public String toString() {
        return "Wind{"
            + "multiplier=" + multiplier
            + ", name=" + name + "}";
    }
}

```

Рисунок 6 - дочерний класс ветер Wind

```

package weathers;

import java.util.Objects;

public class Fog extends Weather { 5 usages  3 elen *
    public Fog() { super( multiplier: 2, name: "Туман"); }

    public Fog(int power) { super( multiplier: 2, name: "Туман", power); }

    @Override 1 usage  3 elen
    void draw() { System.out.println("Туман стелится над болотами"); }

    @Override 4 usages  3 elen *
    public boolean equals(Object ob) {
        if (this == ob) {
            return true;
        }
        if (ob == null || getClass() != ob.getClass()) {
            return false;
        }
        Fog fog = (Fog) ob;
        return Objects.equals(multiplier, fog.multiplier) && Objects.equals(name, fog.name);
    }

    @Override 3 elen *
    public String toString() {
        return "Fog{"
            + "multiplier=" + multiplier
            + ", name=" + name + "}";
    }
}

```

Рисунок 7 - дочерний класс туман Fog

```

package weathers;

import java.util.Objects;

public class Rain extends Weather { 5 usages 2 elen *

    public Rain() { super( multiplier: 3, name: "Дождь"); }

    public Rain(int power) { super( multiplier: 3, name: "Дождь", power); }

    @Override 1 usage 2 elen *
    void draw() {
        System.out.println("Вдруг упала одна капля, а после капля за каплей начали сливать в непреодолимую стену.");
    }

    @Override 4 usages 2 elen *
    public boolean equals(Object ob) {
        if (this == ob) {
            return true;
        }
        if (ob == null || getClass() != ob.getClass()) {
            return false;
        }
        Rain rain = (Rain) ob;
        return Objects.equals(multiplier, rain.multiplier) && Objects.equals(name, rain.name);
    }

    @Override 2 elen *
    public String toString() {
        return "Rain{"
            + "multiplier=" + multiplier
            + ", name=" + name + "}";
    }
}

```

Рисунок 8 - дочерний класс дождь Rain

```

package helps;

public interface Fly { 2 usages 2 implementations
    void rise() throws BirdSleepException; 2 u

    void land(String name); 1 usage 1 implementati
}

```

Рисунок 9 - интерфейс полёт Fly

```
package enums;

public enum Statuses {
    FLY, 1 usage
    SLEEP, 7 usages
    ONSURFACE, 2 usages
}
```

Рисунок 10 – enum статусов для Bird

```
package helps;

public class RandomCount { 4 usages 2 elen *
    private RandomCount() { no usages new *
    }

    static public int GiveCount() { return (int) (Math.random() * 10 + 1); }
}
```

Рисунок 11 - вспомогательный класс для генерирования рандомных чисел

```

package audiences;

import enums.Lenght;

public abstract class Audience { 5 usages 2 inheritors 1 elen *
    private int len; 3 usages

    public Audience(int len) { this.len = len; }

    public void reaction(int admiration) { 1 usage 1 elen *
        if (admiration >= 7) {
            cheers();
        } else if (admiration >= 5) {
            looking();
        } else if (admiration >= 2) {
            inattentivelyLooking();
        }
    }

    protected int getLen(){return len;} 4 usages new *
    protected void setLen(int len) { this.len=len;} no usa

    protected abstract void cheers(); 1 usage 2 implementation
    protected abstract void looking(); 1 usage 2 implementation
    protected abstract void inattentivelyLooking(); 1 usage
}

```

Рисунок 12 - абстрактный класс Аудитория

```

package audiences;

import enums.Lenght;

public class WithHorns extends Audience { 3 usages 2 elen *

    public WithHorns(int len) { 1 usage new *
        super(len);
    }
    protected Lenght determinantSize() { 3 usages new *
        if (getLen() < 25) {
            return Lenght.SMALL;
        } else if (getLen() < 75) {
            return Lenght.NORMAL;
        } else {
            return Lenght.BIG;
        }
    }
}

@Override 1 usage 2 elen *
protected void cheers() {
    System.out.println("Звери наклоняют " + determinantSize().getSize() + " рога в лад танцу");
}

@Override 1 usage 2 elen *
protected void looking() { System.out.println("Звери наклоняют " + determinantSize().getSize() + " рога"); }

@Override 1 usage 2 elen *
protected void inattentivelyLooking() {
    System.out.println("Звери иногда наклоняют " + determinantSize().getSize() + " рога");
}
}

```

Рисунок 13 - дочерний класс животные с рогами

```

package audiences;

import enums.Lenght;

public class WithTail extends Audience { 3 usages 2 elen *

    public WithTail(int lenTail) { super(lenTail); }
    protected Lenght determinantSize() { 3 usages new *
        if (getLen() < 10) {
            return Lenght.SMALL;
        } else if (getLen() < 20) {
            return Lenght.NORMAL;
        } else {
            return Lenght.BIG;
        }
    }

    @Override 1 usage 2 elen *
    protected void cheers() {
        System.out.println("Звери в восхищении бьют " + determinantSize().getSize() + " хвостами");
    }

    @Override 1 usage 2 elen *
    protected void looking() {
        System.out.println("Звери бьют " + determinantSize().getSize() + " хвостами в такт");
    }

    @Override 1 usage 2 elen *
    protected void inattentivelyLooking() {
        System.out.println("Звери, когда видят, бьют " + determinantSize().getSize() + " хвостами");
    }
}

```

Рисунок 14 - дочерний класс животные с хвостом

```

package helps;

public class BirdSleepException extends Exception { 10 usages 2 elen *
    protected String name;

    public BirdSleepException(String name, String message) { 1 usage 2 elen *
        super(message);
        this.name = name;
    }

    @Override 2 usages new *
    public String getMessage() { return name + " не может летать, так как у него недостаточно крыльев("; }
}

```

Рисунок 15 - исключение для птиц, чтобы они не взлетали без крыльев

```
package helps;

public record BirdAppearance( 9
    String color, 1 usage
    String neck, 1 usage
    String size) { 1 usage
}
```

Рисунок 16 – record для описания внешности птиц

```
import audiences.Audience;
import audiences.WithHorns;
import audiences.WithTail;
import birds.Crane;
import enums.Statuses;
import helps.Area;
import helps.BirdCantException;
import helps.BirdSleepException;
import weathers.Fog;
import weathers.Rain;
import weathers.Weather;
import weathers.Wind;

import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) throws BirdSleepException {
        Crane crane1 = new Crane( "Никита", 2, 2, "коричневая", "длинной");
        Crane crane2 = new Crane( "Маша", 2, 1);
        Crane crane3 = new Crane( "Беремот", 1);
        List<Crane> cranes = new ArrayList<>();
        cranes.add(crane1);
        cranes.add(crane2);
        cranes.add(crane3);
        for (Crane crane : cranes) {
            try {
                crane.rise();
            } catch (BirdSleepException e) {
                System.out.println(e.getMessage());
            }
        }

        Wind wind1 = new Wind();
        Rain rain1 = new Rain();
        Fog fog1 = new Fog();
        ArrayList<Weather> weathers = new ArrayList<>();
        weathers.add(wind1);
        weathers.add(rain1);
        weathers.add(fog1);

        for (Crane crane : cranes) {
            for (Weather weather : weathers) {
                if (!crane.getStatus().equals(Statuses.SLEEP)) {
                    if (crane.getStamina() > weather.Hinder()) {
                        crane.useStamina(weather.Hinder());
                        System.out.println(crane.getName() + " преодолел " + weather.name);
                    } else {
                        try {
                            throw new BirdCantException(crane.getName(), " не может преодолеть препятствие");
                        } catch (BirdCantException e) {
                        }
                    }
                }
            }
        }
    }
}
```

Рисунок 17 - класс Main

```

public class Main {
    public static void main(String[] args) throws BirdSleepException {
        for (Crane crane : cranes) {
            for (Weather weather : weathers) {
                if (!crane.getStatus().equals(Statutes.SLEEP)) {
                    if (crane.getStamina() > weather.Hinder()) {
                        crane.useStamina(weather.Hinder());
                        System.out.println(crane.getName() + " преодолел " + weather.name);
                    } else {
                        try {
                            throw new BirdCantException(crane.getName(), " не может преодолеть припятствие");
                        } catch (BirdCantException e) {
                            System.out.println(e.getMessage());
                        } finally {
                            crane.useStamina(-1);
                        }
                    }
                }
            }
        }

        WithHorns horns = new WithHorns(50);
        WithTail tail = new WithTail(10);
        List<Audience> audiences = List.of(horns, tail);

        Area KyLaberga = new Area("Кылабегра");
        for (Crane crane : cranes) {
            KyLaberga.addCrane(crane);
        }

        for (Crane crane : KyLaberga.getCranes()) {
            while (crane.getStamina() > 0 && crane.getAdmiration() <= 7) {
                for (Audience audience : audiences) {
                    audience.reaction(crane.getAdmiration());
                }
                crane.doAction();
                if (crane.getAdmiration() > 7) {
                    System.out.println(crane.getName() + " прекрасно закончил свой танец");
                }
            }
        }

        while (!KyLaberga.getCranes().isEmpty()) {
            KyLaberga.removeCrane(KyLaberga.getCranes().getFirst());
        }
    }
}

```

Рисунок 18 - класс Main 2

```

package helps;

public class BirdCantException extends Exception { 3 usages ⓘ
    protected String name;
    protected String message; 2 usages

    public BirdCantException(String name, String message) { 1
        super(message);
        this.message = message;
        this.name = name;
    }

    @Override 2 usages new *
    public String getMessage() { return name + message; }
}

```

Рисунок 19 - исключение, если птицы не могут победить погоду

```

package enums;

public enum Lenght { 11 usages ⓘ
    BIG( size: "большими"), 2 usages
    NORMAL( size: "средними"), 2 u
    ⚡ SMALL( size: "маленькими"); 2
    private final String size;

    Lenght(String size) { 6 usages
        this.size = size;
    }

    public String getSize() { 6
        return size;
    }
}

```

Рисунок 20 – enum для размеров

Метод от ИИ:

Промпт:

Напиши полную реализацию метода rise() на java с правильной обработкой исключений и логикой. Класс Bird - абстрактный класс птиц, поля класса: wingsCounter (количество крыльев, 0-2) stamina, status: Statuses (enum: FLY, SLEEP, ONSURFACE) height, name. Метод должен: Проверять, что wingsCounter >= 2, если крыльев меньше 2 - бросать BirdSleepException, тратить выносливость (10% от текущей stamina), менять status на FLY, увеличивать height на 50, выводить сообщение о взлете, проверять stamina после взлета.

GigaChat:

```
public void rise() throws BirdSleepException {

    if (wingsCounter < 2) {
        throw new BirdSleepException("Невозможно взлететь: недостаточно крыльев");
    }

    if (status.equals(Status.SLEEP)) {
        System.out.println(name + ": Я сплю и не могу взлететь.");
        return;
    }

    if (status.equals(Status.FLY)) {
        System.out.println(name + ": Уже в полёте!");
        return;
    }

    // Уменьшаем выносливость на 10%
    double decreaseAmount = stamina * STAMINA_DECREASE_PERCENT;
    stamina -= decreaseAmount;
    System.out.println(name + ": Потрачено " + Math.round(decreaseAmount) +
"% выносливости.");
```

```
// Проверяем остаток выносливости
if (stamina <= 0) {
    System.out.println(name + ": Выносливость закончилась, птица садится
обратно.");
    status = Status.ONSURFACE;
    return;
}

// Меняем статус на полет
status = Status.FLY;
height += HEIGHT_INCREASE_AMOUNT;
System.out.println(name + ": Взлетел! Высота увеличилась до " + height);

}
```

Qwen:

```
public void rise() {
    // 1. Проверка: достаточно ли крыльев для взлёта
    if (wingsCounter < 2) {
        throw new BirdSleepException(
            String.format("Bird '%s' cannot rise: only %d wing(s) available. Need 2 to fly.",
            name, wingsCounter)
        );
    }

    // 2. Проверка: жив ли птица? (дополнительная защита — не по ТЗ, но
    разумно)
    if (stamina <= 0) {
```

```
throw new IllegalStateException(  
String.format("Bird '%s' has no stamina left and cannot rise.", name)  
);  
}
```

```
// 3. Тратим 10% текущей выносливости
```

```
double staminaCost = stamina * 0.1;
```

```
stamina -= staminaCost;
```

```
// 4. Меняем статус на FLY и увеличиваем высоту на 50
```

```
status = Statuses.FLY;
```

```
height += 50.0;
```

```
// 5. Выводим сообщение о взлёте
```

```
System.out.printf(  
"[INFO] Bird '%s' has risen! Height: %.1f, Stamina after rise: %.2f (cost:  
%.2f)%n",  
name, height, stamina, staminaCost  
);
```

```
// 6. Проверяем stamina после взлёта: если <= 0 — птица падает (логика по  
усмотрению)
```

```
if (stamina <= 0) {
```

```
System.err.printf("[WARNING] Bird '%s' exhausted all stamina after  
rising!%n", name);
```

```
// Можно дополнительно сменить статус, но по ТЗ не требуется — только  
"проверить"
```

```
}  
}
```

DeepSeek:

```
public void rise() throws BirdSleepException {  
    try {  
        // Проверка количества крыльев  
        if (wingsCounter < 2) {  
            throw new BirdSleepException("Птица " + name + " не может  
взлететь: недостаточно крыльев (только " + wingsCounter + ")");  
        }  
  
        // Проверка текущего статуса (опционально, но логично)  
        if (status == Statuses.FLY) {  
            System.out.println(name + " уже летит на высоте " + height);  
            return;  
        }  
  
        // Сохранение предыдущего статуса для логирования  
        Statuses previousStatus = status;  
  
        // Трата выносливости (10% от текущей stamina)  
        double staminaCost = stamina * 0.1;  
        stamina -= staminaCost;  
  
        // Изменение статуса на FLY  
        status = Statuses.FLY;  
  
        // Увеличение высоты на 50  
        height += 50;  
  
        // Вывод сообщения о взлете
```

```

        System.out.println(name + " взлетела! " +
            "Предыдущий статус: " + previousStatus + ", " +
            "Текущая высота: " + height + ", " +
            "Потрачено выносливости: " + String.format("%.1f",
staminaCost));

        // Проверка stamina после взлета
        if (stamina <= 0) {
            System.out.println("Внимание! У " + name + " закончилась
выносливость!");
            stamina = 0;
        } else if (stamina < 10) {
            System.out.println("Предупреждение: у " + name + " низкая
выносливость (" +
                String.format("%.1f", stamina) + ")");
        }

    } catch (BirdSleepException e) {
        // Перебрасываем проверенное исключение
        throw e;
    } catch (Exception e) {
        // Обработка непредвиденных исключений
        System.err.println("Неожиданная ошибка при взлете " + name + ": " +
e.getMessage());
        // Можно выбросить исключение дальше или обработать иначе
        throw new RuntimeException("Ошибка при выполнении взлета", e);
    }
}

// Вспомогательный метод для безопасного взлета (без выбрасывания
исключения)
public boolean tryRise() {
    try {
        rise();
        return true;
    } catch (BirdSleepException e) {
        System.out.println(e.getMessage());
    }
}

```

```

        return false;
    } catch (Exception e) {
        System.err.println("Критическая ошибка: " + e.getMessage());
        return false;
    }
}

}

}

```

Анализ:

GigaChat

Плюсы:

Хорошая структура кода

Константы вместо магических чисел

Проверяет все статусы птицы

Минусы:

Неправильный конструктор исключения (нужно 2 параметра)

Не использует существующие методы (useStamina, checkStamina)

Изменил типы данных (stamina: int → double)

Qwen

Плюсы:

Профессиональное логирование [INFO], [WARNING]

Подробные сообщения об ошибках

Хорошие комментарии

Минусы:

Фатальная ошибка: нет throws BirdSleepException в сигнатуре

Вводит ненужные новые исключения

Полностью игнорирует архитектуру проекта

DeepSeek

Плюсы:

Правильная сигнатура метода

Дополнительный метод `tryRise()` - умная идея

Сохраняет предыдущий статус для логирования

Минусы:

Неправильный конструктор исключения

Избыточный `try-catch`

Не использует готовые методы проекта

Заключение

Для работы были изучены принципы объектно-ориентированного программирования SOLID и STUPID, класс Object и реализация его методов по умолчанию, наследование, абстрактные классы, а также отличие абстрактного класса от интерфейса, перечисляемый тип данных enum, запись record, и особенности их использования, перегрузка и переопределение методов, обработка исключений, массивы стандартные и динамические, а также вложенный, локальные и анонимные классы.